```java

public class MultithreadedMatrixMultiplication {

    static class RowThread extends Thread {
        int[][] a, b, result;
        int row;

        RowThread(int[][] a, int[][] b, int[][] result, int row) {
            this.a = a;
            this.b = b;
            this.result = result;
            this.row = row;
        }

        public void run() {
            for (int j = 0; j < b[0].length; j++) {
                for (int k = 0; k < b.length; k++) {
                    result[row][j] += a[row][k] * b[k][j];
                }
            }
        }
    }

    static void displayMatrix(int[][] matrix) {
        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[0].length; j++) {
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
        }
    }

    public static void main(String[] args) throws InterruptedException {

        int[][] a = {
            {2, 1, 3},
            {4, 0, 2}
        };

        int[][] b = {
            {5, 2},
            {1, 4},
            {3, 7}
        };

        if (a[0].length != b.length) {
```

```
            System.out.println("Matrix multiplication is not possible.");
            return;
        }


        int rows = a.length;
        int cols = b[0].length;

        int[][] normalResult = new int[rows][cols];
        int[][] threadResult = new int[rows][cols];

        System.out.println("Matrix A:");
        displayMatrix(a);

        System.out.println("\nMatrix B:");
        displayMatrix(b);

        long startTime = System.nanoTime();

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                for (int k = 0; k < b.length; k++) {
                    normalResult[i][j] += a[i][k] * b[k][j];
                }
            }
        }

        double normalTime =
                (System.nanoTime() - startTime) / 1000000.0;

        startTime = System.nanoTime();

        RowThread[] threads = new RowThread[rows];

        for (int i = 0; i < rows; i++) {
            threads[i] = new RowThread(a, b, threadResult, i);
            threads[i].start();
        }

        for (int i = 0; i < rows; i++) {
            threads[i].join();
        }

        double threadTime =
                (System.nanoTime() - startTime) / 1000000.0;

        System.out.println("\nNormal Multiplication Result:");
        displayMatrix(normalResult);
```

```
94
95          System.out.println("\nMultithreaded Multiplication Result:");
96          displayMatrix(threadResult);
97
98          System.out.println("\nPerformance Comparison:");
99          System.out.println("Normal execution time: "
100                 + normalTime + " ms");
```

```
101         System.out.println("Multithreaded execution time: "
102                 + threadTime + " ms");
103
104         System.out.println("\nBoth results are "
105                 + (java.util.Arrays.deepEquals(normalResult, threadResult)
106                 ? "identical." : "different."));
107     }
108 }
109
```

Output :

```
PS D:\Assignment_06_Mini_Project_1_Multithreaded_Matrix_Multiplication> javac MultithreadedMatrixMultiplication.java
PS D:\Assignment_06_Mini_Project_1_Multithreaded_Matrix_Multiplication> java MultithreadedMatrixMultiplication
Matrix A:
2 1 3
4 0 2

Matrix B:
5 2
1 4
3 7

Normal Multiplication Result:
20 29
26 22

Multithreaded Multiplication Result:
20 29
26 22

Performance Comparison:
Normal execution time: 0.0028 ms
Multithreaded execution time: 4.1478 ms

Both results are identical.
PS D:\Assignment_06_Mini_Project_1_Multithreaded_Matrix_Multiplication>
```